Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 8

Дисциплина: МДК 03.01

Тема: Портфолио сопровождения системы заявок

Вариант № 2

Студент: Шваб Роберт Эдуардович Подпись _____

Группа: И13_24_11

Руководитель: _____ Подпись _____

Дата представления: «___» _____ 2026 г.

Оценка: _____

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ.....3
1.1 Цель и исходные данные.....3
1.2 Модули портфолио.....3
1.3 Результаты прогона.....4
1.4 Программа и воспроизведение результата.....4
1.5 Выводы.....5
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6
ПРИЛОЖЕНИЕ А.....7
ПРИЛОЖЕНИЕ Б.....16

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ

1.1 Цель и исходные данные

Цель работы — объединить обновление, офлайн-доставку, проверки защиты, резервирование и протокол SAT в воспроизводимый комплект.

Постановка работы приведена в перечне заданий МДК 03.01 [1].

Исходники и отчёты: репозиторий MDK0301-ServiceDesk на GitHub

1.2 Модули портфолио

Таблица 1 сопоставляет компоненты и проверяемые результаты. На рисунке 1 показаны связи между API, локальной очередью и средствами сопровождения.

Таблица 1 — Модули портфолио и проверки

Модуль	Технология	Контроль
Updater	C#, SemVer, SHA-256	Установка, повтор, отказ по хешу
OfflineBuffer	EF Core, SQLite, Polly	Событие сохраняется и доставляется
SecurityDemo	BCrypt, X.509, RBAC, DPAPI	Разрешение и отказ; DPAPI на Windows
Operations	C#, ZIP, HTTP	Архив, хеш, протокол SAT

Источник: составлено по материалам практической работы.

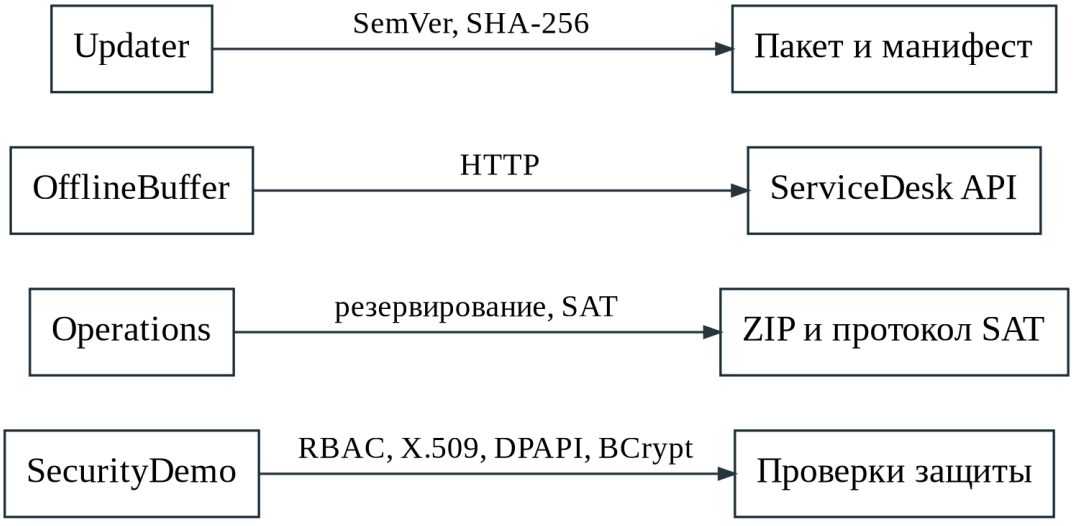


Рисунок 1 — Компоненты портфолио сопровождения

Updater сравнивает версии SemVer [2], проверяет SHA-256 временной копии пакета и сохраняет номер установленной версии. Проверен повторный

запуск и отказ при изменении файла. Хеш защищает целостность относительно доверенного манифеста, но сам по себе не подтверждает его происхождение.

OfflineBuffer сохраняет заявку в SQLite через EF Core, повторяет временно неудачный HTTP-запрос через Polly и помечает событие отправленным после успешного ответа. При остановленном сервере событие осталось в очереди; после запуска API доставлено.

SecurityDemo проверяет корректный и ошибочный пароль через BCrypt, разрешения ролей, цепочку и имя X.509. Проверка использует временный тестовый центр, не меняет системные сертификаты. DPAPI защищает данные в контексте текущего пользователя Windows [3]; результат записывается отдельной строкой в журнале.

1.3 Результаты прогона

Фактический вывод программы приведён в приложении Б. Запуск из Visual Studio: проект Verification, профиль ПР 8, Ctrl+F5.

Operations создаёт ZIP и файл SHA-256, а также протокол SAT в Markdown. Скрипты PowerShell запускают эти команды; реализация находится на C#. В Evidence сохранены журналы, образец архива и стендовый SAT.

Проверка DPAPI выполняется на Windows. Ссылка на отдельный прогон Windows: https://github.com/rob4ik72-del/MDK0301-ServiceDesk/actions/runs/4268485177 9

1.4 Программа и воспроизведение результата

Профиль «ПР 8» выполняет модули обновления, очереди, защиты и резервирования. Сохранение без сети и отказ изменённому пакету являются ожидаемыми сценариями; их коды проверяются запускателем Verification.

Перед показом выполняется «Сборка → Пересобрать решение». Результат сборки виден в окне «Вывод → Сборка», а результат программы — в отдельной консоли Visual Studio.

1.5 Выводы

Портфолио собрано в одном решении. По журналам можно отличить успешные сценарии от ожидаемых отказов и повторить проверку на своём ПК.

Листинги приведены в приложении А.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 МДК 03.01. Перечень практических работ № 1–8. Учебное задание.

2 Semantic Versioning 2.0.0. https://semver.org/

3 Microsoft Learn. DataProtectionScope Enum. https://learn.microsoft.com/dotnet/api/system.security.cryptography.dataprotectionscope

ПРИЛОЖЕНИЕ А

Исходный код программы

Code/Updater/Program.cs

```
using System.Security.Cryptography;
using System.Text.Json;
using System.Text.RegularExpressions;

try
{
   if (args.Length != 3) throw new ArgumentException("Usage: Updater manifest.json package-file target-
file");
   var manifest = JsonSerializer.Deserialize<Manifest>(File.ReadAllText(args[0]),
      new JsonSerializerOptions { PropertyNameCaseInsensitive = true }) ?? throw new
FormatException("Empty manifest");
   var next = SemVersion.Parse(manifest.Version);
   var target = Path.GetFullPath(args[2]);
   var state = target + ".version";
   var current = SemVersion.Parse(File.Exists(state) ? File.ReadAllText(state).Trim() : "1.0.0");
   if (next.CompareTo(current) <= 0) { Console.WriteLine("Already current or newer"); return 0; }
   Directory.CreateDirectory(Path.GetDirectoryName(target)!);
   var temp = target + "." + Guid.NewGuid().ToString("N") + ".new";
   var stagedState = temp + ".version";
   try
   {
      File.Copy(args[1], temp);
      using (var input = File.OpenRead(temp))
      {
         var actual = Convert.ToHexString(SHA256.HashData(input));
         if (!actual.Equals(manifest.Sha256, StringComparison.OrdinalIgnoreCase))
            throw new InvalidDataException("SHA-256 mismatch; update rejected");
      }
      File.WriteAllText(stagedState, manifest.Version);
      File.Move(temp, target, overwrite: true);
      File.Move(stagedState, state, overwrite: true);
      Console.WriteLine($"Updated to {manifest.Version}; SHA-256 verified");
   }
   finally { File.Delete(temp); File.Delete(stagedState); }
   return 0;
}
catch (Exception ex) when (ex is IOException or InvalidDataException or FormatException or JsonException
or ArgumentException
   or UnauthorizedAccessException)
{
   Console.Error.WriteLine(ex.Message);
   return 1;
}
record Manifest(string Version, string Sha256);

sealed class SemVersion(string[] core, string[] pre) : IComparable<SemVersion>
{
   private readonly string[] numbers = core;
   private readonly string[] qualifiers = pre;
   public static SemVersion Parse(string text)
   {
```

```
      var m = Regex.Match(text, @"\A(0|[1-9][0-9]*)\.(0|[1-9][0-9]*)\.(0|[1-9][0-9]*)(?:-([0-9A-Za-z-]+(?:\.[0-9A-Za-z-]+)*))?(?:\+[0-9A-Za-z-]+(?:\.[0-9A-Za-z-]+)*)?\z");
      if (!m.Success) throw new FormatException("Invalid SemVer");
      var pre = m.Groups[4].Success ? m.Groups[4].Value.Split('.') : [];
      if (pre.Any(x => Numeric(x) && x.Length > 1 && x[0] == '0')) throw new FormatException("Invalid SemVer prerelease");
      return new SemVersion([m.Groups[1].Value, m.Groups[2].Value, m.Groups[3].Value], pre);
   }
   public int CompareTo(SemVersion? other)
   {
      if (other is null) return 1;
      for (var i = 0; i < 3; i++)
      {
         var cmp = CompareNumeric(numbers[i], other.numbers[i]);
         if (cmp != 0) return cmp;
      }
      if (qualifiers.Length == 0 || other.qualifiers.Length == 0)
         return (qualifiers.Length == 0).CompareTo(other.qualifiers.Length == 0);
      for (var i = 0; i < Math.Min(qualifiers.Length, other.qualifiers.Length); i++)
      {
         var a = qualifiers[i]; var b = other.qualifiers[i];
         var cmp = Numeric(a) && Numeric(b) ? CompareNumeric(a, b) :
            Numeric(a) != Numeric(b) ? (Numeric(a) ? -1 : 1) : string.CompareOrdinal(a, b);
         if (cmp != 0) return cmp;
      }
      return qualifiers.Length.CompareTo(other.qualifiers.Length);
   }
   private static bool Numeric(string s) => s.All(char.IsAsciiDigit);
   private static int CompareNumeric(string a, string b) =>
      a.Length == b.Length ? string.CompareOrdinal(a, b) : a.Length.CompareTo(b.Length);
}
```

Code/OfflineBuffer/Program.cs

```
using System.Net;
using System.Net.Http.Json;
using Microsoft.EntityFrameworkCore;
using Polly;

var address = args.FirstOrDefault() ?? "http://127.0.0.1:5000";
var file = args.ElementAtOrDefault(1) ?? "offline.db";
using var db = new LocalDb(file);
await db.Database.EnsureCreatedAsync();
if (args.ElementAtOrDefault(2) is { } title)
{
   var item = new PendingEvent { EventId = Guid.NewGuid(), Title = title };
   db.PendingEvents.Add(item);
   await db.SaveChangesAsync();
   Console.WriteLine($"Queued {item.EventId}");
}
using var http = new HttpClient { BaseAddress = new Uri(address), Timeout = TimeSpan.FromSeconds(2) };
var retry = Policy.Handle<HttpRequestException>(e => e.StatusCode is null or >= HttpStatusCode.InternalServerError)
   .Or<TaskCanceledException>().WaitAndRetryAsync(3, i => TimeSpan.FromMilliseconds(150 * i),
      (e, delay, i, _) => Console.WriteLine($"Retry {i}: {e.GetType().Name}"));
foreach (var item in await db.PendingEvents.Where(e => !e.Sent).ToListAsync())
{
   try
   {
      await retry.ExecuteAsync(async () =>
```

```
      {
         using var response = await http.PostAsJsonAsync("/api/v2/sync", new { item.EventId, item.Title });
         response.EnsureSuccessStatusCode();
      });
      item.Sent = true;
      await db.SaveChangesAsync();
      Console.WriteLine($"Synchronized {item.EventId}");
   }
   catch (Exception e) when (e is HttpRequestException or TaskCanceledException)
   {
      Console.WriteLine($"Retained {item.EventId}: {e.GetType().Name}");
   }
}
var pending = await db.PendingEvents.CountAsync(e => !e.Sent);
Console.WriteLine($"Pending: {pending}");
return pending == 0 ? 0 : 2;

class PendingEvent
{
   public int Id { get; set; }
   public Guid EventId { get; set; }
   public string Title { get; set; } = "";
   public bool Sent { get; set; }
}
class LocalDb(string file) : DbContext
{
   public DbSet<PendingEvent> PendingEvents => Set<PendingEvent>();
   protected override void OnConfiguring(DbContextOptionsBuilder options) =>
      options.UseSqlite(new Microsoft.Data.Sqlite.SqliteConnectionStringBuilder { DataSource =
file }.ToString());
   protected override void OnModelCreating(ModelBuilder builder) =>
      builder.Entity<PendingEvent>().HasIndex(e => e.EventId).IsUnique();
}
```

Code/SecurityDemo/Program.cs

```
using System.Security.Cryptography;
using System.Security.Cryptography.X509Certificates;
using System.Text;

var sample = "local training value";
var hash = BCrypt.Net.BCrypt.HashPassword(sample, workFactor: 12);
Check(BCrypt.Net.BCrypt.Verify(sample, hash) && !BCrypt.Net.BCrypt.Verify("wrong", hash),
   "BCrypt: valid password accepted, wrong one rejected");
var rights = new Dictionary<string, string[]> {
   ["viewer"] = ["read"], ["operator"] = ["read", "create"], ["admin"] = ["read", "create", "delete"]
};
bool Allowed(string role, string action) => rights.TryGetValue(role, out var actions) &&
actions.Contains(action);
Check(Allowed("operator", "create") && !Allowed("operator", "delete") &&
   Allowed("admin", "delete") && !Allowed("unknown", "read"), "RBAC: allow and deny cases");

using var rootKey = RSA.Create(2048);
using var leafKey = RSA.Create(2048);
var now = DateTimeOffset.UtcNow;
var rootRequest = new CertificateRequest("CN=ServiceDesk Test CA", rootKey,
   HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
rootRequest.CertificateExtensions.Add(new X509BasicConstraintsExtension(true, false, 0, true));
rootRequest.CertificateExtensions.Add(new X509KeyUsageExtension(X509KeyUsageFlags.KeyCertSign, true));
using var root = rootRequest.CreateSelfSigned(now.AddDays(-3), now.AddDays(4));
```

```
var leafRequest = new CertificateRequest("CN=servicedesk.test", leafKey,
   HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
leafRequest.CertificateExtensions.Add(new X509BasicConstraintsExtension(false, false, 0, true));
leafRequest.CertificateExtensions.Add(new X509EnhancedKeyUsageExtension(new OidCollection
{ new("1.3.6.1.5.5.7.3.1") }, true));
var names = new SubjectAlternativeNameBuilder();
names.AddDnsName("servicedesk.test");
leafRequest.CertificateExtensions.Add(names.Build());
using var leaf = leafRequest.Create(root, now.AddHours(-1), now.AddDays(1),
RandomNumberGenerator.GetBytes(16));
Check(Trusted(leaf, root, now) && leaf.MatchesHostname("servicedesk.test", false, false),
   "X.509: valid chain and hostname accepted");
Check(!leaf.MatchesHostname("other.test", false, false) && !Trusted(leaf, root, now.AddDays(2)),
   "X.509: wrong hostname and expired certificate rejected");

if (OperatingSystem.IsWindows())
{
   var bytes = Encoding.UTF8.GetBytes(sample);
   var encrypted = ProtectedData.Protect(bytes, null, DataProtectionScope.CurrentUser);
   var decoded = ProtectedData.Unprotect(encrypted, null, DataProtectionScope.CurrentUser);
   Check(bytes.SequenceEqual(decoded) && !bytes.SequenceEqual(encrypted), "DPAPI: current-user
roundtrip");
}
else Console.WriteLine("SKIP: DPAPI requires Windows");

static bool Trusted(X509Certificate2 leaf, X509Certificate2 root, DateTimeOffset at)
{
   using var chain = new X509Chain();
   chain.ChainPolicy.TrustMode = X509ChainTrustMode.CustomRootTrust;
   chain.ChainPolicy.CustomTrustStore.Add(root);
   // This generated CA exists only for this test and has no revocation service.
   chain.ChainPolicy.RevocationMode = X509RevocationMode.NoCheck;
   chain.ChainPolicy.VerificationTime = at.UtcDateTime;
   chain.ChainPolicy.ApplicationPolicy.Add(new Oid("1.3.6.1.5.5.7.3.1"));
   return chain.Build(leaf);
}
static void Check(bool passed, string message)
{
   if (!passed) throw new InvalidOperationException("FAIL: " + message);
   Console.WriteLine("PASS: " + message);
}
```

Code/Operations/Program.cs

```
using System.IO.Compression;
using System.Net;
using System.Security.Cryptography;
using System.Text.Json;
using Microsoft.Data.Sqlite;

try
{
   switch (args)
   {
      case ["manifest", var version, var file, var output]:
         File.WriteAllText(output, JsonSerializer.Serialize(new { Version = version, Sha256 = Hash(file) }));
         Console.WriteLine("Manifest written: " + output);
         return 0;
      case ["backup", var source, var destination]:
```

```
        if
(Path.GetFullPath(destination).StartsWith(Path.TrimEndingDirectorySeparator(Path.GetFullPath(source)) +
Path.DirectorySeparatorChar,
            OperatingSystem.IsWindows() ? StringComparison.OrdinalIgnoreCase : StringComparison.Ordinal))
            throw new ArgumentException("Archive must be outside source directory");
        ZipFile.CreateFromDirectory(source, destination);
        File.WriteAllText(destination + ".sha256", Hash(destination));
        using (var zip = ZipFile.OpenRead(destination))
            Console.WriteLine($"Backup: {zip.Entries.Count} entries; SHA-256 {Hash(destination)}");
        return 0;
    case ["sat", var address, var output]:
        return await Sat(address, output);
    case ["migration", var directory, var database]:
        return Migration(directory, database);
    default:
        Console.Error.WriteLine("Operations: manifest VERSION FILE JSON | backup SOURCE ZIP | sat URL
MARKDOWN | migration SQL_DIR DB");
        return 1;
  }
}
catch (Exception error)
{
  Console.Error.WriteLine(error.Message);
  return 1;
}

static string Hash(string path)
{
  using var stream = File.OpenRead(path);
  return Convert.ToHexString(SHA256.HashData(stream));
}
static async Task<int> Sat(string address, string output)
{
  using var http = new HttpClient { BaseAddress = new Uri(address), Timeout = TimeSpan.FromSeconds(5) };
  var checks = new List<(string Scenario, bool Passed, string Detail)>();
  foreach (var (route, expected) in new[]
  {
    ("/health", HttpStatusCode.OK),
    ("/api/v1/tickets", HttpStatusCode.OK),
    ("/api/v2/tickets", HttpStatusCode.OK),
    ("/api/v1/tickets/2147483647", HttpStatusCode.NotFound)
  })
  {
    try
    {
      using var response = await http.GetAsync(route);
      var correctBody = true;
      if (expected != HttpStatusCode.NotFound)
      {
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        correctBody = route == "/health" ? json.RootElement.GetProperty("status").GetString() == "ok"
          : json.RootElement.ValueKind == JsonValueKind.Array;
      }
      checks.Add((route, response.StatusCode == expected && correctBody, $"HTTP
{(int)response.StatusCode}"));
    }
    catch (Exception e) when (e is HttpRequestException or TaskCanceledException or JsonException or
KeyNotFoundException or InvalidOperationException)
    { checks.Add((route, false, e.GetType().Name)); }
```

```
    }
    foreach (var x in checks) Console.WriteLine($"{(x.Passed ? "PASS" : "FAIL")}: GET {x.Scenario} —
{x.Detail}");
    var passed = checks.Count(x => x.Passed);
    var lines = new List<string> { "# Протокол SAT — стенд", "", $"Дата: {DateTimeOffset.UtcNow:yyyy-MM-
dd HH:mm:ss} UTC", "",
        "| Сценарий | Результат | Ответ |", "| --- | --- | --- |" };
    foreach (var x in checks) lines.Add($"| GET {x.Scenario} | {(x.Passed ? "PASS" : "FAIL")} | {x.Detail} |");
    lines.Add($"\nПройдено: {passed}/{checks.Count}. Согласование заказчиком проводится отдельно.");
    File.WriteAllLines(output, lines);
    Console.WriteLine($"SAT: {passed}/{checks.Count} PASS");
    return passed == checks.Count ? 0 : 2;
}
static int Migration(string directory, string database)
{
    if (File.Exists(database)) throw new ArgumentException("Use a new database file for the migration
demo");
    using var connection = new SqliteConnection(new SqliteConnectionStringBuilder { DataSource =
database }.ToString());
    connection.Open();
    void Exec(string sql) { using var cmd = connection.CreateCommand(); cmd.CommandText = sql;
cmd.ExecuteNonQuery(); }
    long Read(string sql) { using var cmd = connection.CreateCommand(); cmd.CommandText = sql; return
Convert.ToInt64(cmd.ExecuteScalar()); }
    Exec(File.ReadAllText(Path.Combine(directory, "schema_v1.sql")));
    Exec("INSERT INTO tickets (title,status,legacy_category) VALUES ('Existing ticket','Open','Hardware')");
    Exec(File.ReadAllText(Path.Combine(directory, "migrate_expand.sql")));
    Exec("INSERT INTO tickets (title,status) VALUES ('Old client ticket','Open')");
    if (Read("SELECT COUNT(*) FROM tickets WHERE priority=3") != 2) throw new
InvalidOperationException("Expand failed");
    Console.WriteLine("PASS: original row preserved and v1 insert works after Expand");
    Console.WriteLine("After Expand: 2 rows, priority=3; INSERT v1 works.");
    Exec(File.ReadAllText(Path.Combine(directory, "migrate_contract.sql")));
    if (Read("SELECT COUNT(*) FROM tickets WHERE priority=3") != 2 ||
        Read("SELECT COUNT(*) FROM pragma_table_info('tickets') WHERE name='legacy_category'") != 0)
        throw new InvalidOperationException("Contract failed");
    Console.WriteLine("PASS: Contract removes the legacy column and keeps both rows");
    Console.WriteLine("After Contract: 2 rows; legacy_category absent.");
    return 0;
}
```

Code/Verification/Program.cs

```
using System.Diagnostics;
using System.IO.Compression;
using System.Net;
using System.Net.Sockets;
using System.Security.Cryptography;
using System.Text.Json;

Console.OutputEncoding = System.Text.Encoding.UTF8;
var selected = args is ["--pr", var value] ? value : "all";
if (selected != "all" && selected is not ("04" or "05" or "06" or "07" or "08"))
    throw new ArgumentException("Select PR 04, 05, 06, 07 or 08");
var root = args.Length == 0 || args[0] == "--pr" ? FindRoot() : Path.GetFullPath(args[0]);
var config = new DirectoryInfo(AppContext.BaseDirectory).Parent!.Name;
Console.WriteLine($"МДК 03.01 — ПР {selected}; конфигурация {config}");
var output = Path.GetFullPath(args.Length == 0 || args[0] == "--pr" ? Path.Combine(root,"RunResults") :
args.ElementAtOrDefault(1) ?? Path.Combine(root,"RunResults"));
Directory.CreateDirectory(output);
```

```
var staging = Path.Combine(Path.GetTempPath(), "ServiceDesk-" + Guid.NewGuid().ToString("N"));
Directory.CreateDirectory(staging);
var groups = new Dictionary<string, List<(string Label, string Output)>>();
var total = 0;
var port = GetFreePort();
var address = $"http://127.0.0.1:{port}";
Process? server = null;
try
{
   if (selected is "08" or "all") await Run("08", "Сохранение заявки без сети", "OfflineBuffer", 2, "Pending: 1",
      address, Path.Combine(staging, "offline.db"), "Офлайн заявка");
   server = Start("ServiceDeskApi", ["--urls", address, "--Logging:LogLevel:Default", "Warning"]);
   var discardOut = server.StandardOutput.ReadToEndAsync();
   var discardErr = server.StandardError.ReadToEndAsync();
   using var http = new HttpClient { Timeout = TimeSpan.FromSeconds(3) };
   var ready = false;
   for (var i = 0; i < 60; i++)
   {
      if (server.HasExited) throw new Exception(await discardErr);
      try { ready = (await http.GetAsync(address + "/health")).IsSuccessStatusCode; } catch
(HttpRequestException) { }
      if (ready) break;
      await Task.Delay(150);
   }
   if (!ready) throw new Exception("API did not start");
   if (selected is "04" or "all") await Run("04", "Обратная совместимость API", "ApiCompatibilityTests", 0,
"duplicate event", address);
   File.WriteAllText(Path.Combine(output, "api-v1.json"), await http.GetStringAsync(address +
"/api/v1/tickets"));
   File.WriteAllText(Path.Combine(output, "api-v2.json"), await http.GetStringAsync(address +
"/api/v2/tickets"));
   if (selected is "08" or "all")
   {
   await Run("08", "Отправка после восстановления соединения", "OfflineBuffer", 0, "Pending: 0",
      address, Path.Combine(staging, "offline.db"));
   if (!(await http.GetStringAsync(address + "/api/v2/tickets")).Contains("Офлайн заявка") &&
      !JsonDocument.Parse(await http.GetStringAsync(address + "/api/v2/tickets")).RootElement
         .EnumerateArray().Any(t => t.GetProperty("title").GetString() == "Офлайн заявка"))
      throw new Exception("Offline event missing on server");
   }
   if (selected is "05" or "all") await Run("05", "Миграция SQLite", "Operations", 0, "keeps both rows",
      "migration", Path.Combine(root, "Code", "sample-data"), Path.Combine(staging, "migration.db"));
   if (selected is "07" or "all") await Run("07", "Стендовые проверки SAT", "Operations", 0, "4/4 PASS",
      "sat", address, Path.Combine(output, "SAT.md"));
   if (selected is "06" or "all") await Run("06", "Обмен JSON и XML", "CalsExchange", 0, "CALS: 5/5 PASS",
Path.Combine(output, "cals"));
   if (selected is "08" or "all")
   {
   var pack = Path.Combine(staging, "package.txt");
   var manifest = Path.Combine(staging, "manifest.json");
   var target = Path.Combine(staging, "installed.txt");
   File.WriteAllText(pack, "ServiceDesk release 1.1.0");
   await Run("08", "Формирование манифеста", "Operations", 0, "Manifest written", "manifest", "1.1.0",
pack, manifest);
   await Run("08", "Установка версии 1.1.0", "Updater", 0, "Updated to 1.1.0", manifest, pack, target);
   await Run("08", "Повторный запуск без переустановки", "Updater", 0, "Already current", manifest, pack,
target);
   await Run("08", "Манифест для версии 1.2.0", "Operations", 0, "Manifest written", "manifest", "1.2.0", pack,
manifest);
```

```
    File.AppendAllText(pack, " modified");
    await Run("08", "Отклонение изменённого пакета", "Updater", 1, "SHA-256 mismatch", manifest, pack,
target);
    if (File.ReadAllText(target) != "ServiceDesk release 1.1.0" || File.ReadAllText(target + ".version") != "1.1.0")
        throw new Exception("Rejected update modified installed files");
    await Run("08", "Проверки защиты", "SecurityDemo", 0,
        OperatingSystem.IsWindows() ? "PASS: DPAPI" : "SKIP: DPAPI");
    var archive = Path.Combine(output, "backup.zip");
    File.Delete(archive);
    await Run("08", "Резервное копирование", "Operations", 0, "SHA-256",
        "backup", Path.Combine(root, "Code", "sample-data"), archive);
    using var zip = ZipFile.OpenRead(archive);
    if (zip.GetEntry("schema_v1.sql") is null ||
        Convert.ToHexString(SHA256.HashData(File.ReadAllBytes(archive))) != File.ReadAllText(archive +
".sha256"))
        throw new Exception("Backup validation failed");
    }
    foreach (var (number, runs) in groups)
    {
        var body = string.Join("", runs.Select(x => $"<h2>{H(x.Label)}</h2><pre>{H(x.Output)}</pre>"));
        File.WriteAllText(Path.Combine(output, $"PR{number}-run.html"), $$"""
            <!doctype html><html lang="ru"><meta charset="utf-8"><title>ПР {{number}} — запуск</title>
            <style>body{margin:32px;font:18px Arial;color:#132a40;background:#f4f7fb}
            h1{font-size:30px} h2{font-size:19px;margin:14px 0 5px}
            pre{white-space:pre-wrap;overflow-wrap:anywhere;font:16px monospace;background:white;border-
left:4px solid #2b6ea5;padding:10px;margin:0}</style>
            <h1>Служба заявок · ПР {{number}}</h1><p>Вывод проверочного запуска C#</p>{{body}}</html>
            """);
    }
    Console.WriteLine($"Completed {total} command scenarios; expected exit codes and outputs verified.");
    return 0;
}
finally
{
    if (server is not null)
    {
        if (!server.HasExited) { server.Kill(entireProcessTree: true); await server.WaitForExitAsync(); }
        server.Dispose();
    }
    Directory.Delete(staging, recursive: true);
}

Process Start(string project, string[] command)
{
    var dll = Path.Combine(root, "Code", project, "bin", config, "net8.0", project + ".dll");
    var start = new ProcessStartInfo(Environment.GetEnvironmentVariable("DOTNET_HOST_PATH") ??
"dotnet")
    { WorkingDirectory = staging, RedirectStandardOutput = true, RedirectStandardError = true,
UseShellExecute = false };
    start.ArgumentList.Add(dll);
    foreach (var argument in command) start.ArgumentList.Add(argument);
    return Process.Start(start)!;
}
async Task Run(string number, string label, string project, int exit, string required, params string[]
command)
{
    using var p = Start(project, command);
    var stdout = p.StandardOutput.ReadToEndAsync();
    var stderr = p.StandardError.ReadToEndAsync();
```

```
    try { await p.WaitForExitAsync(new CancellationTokenSource(TimeSpan.FromSeconds(45)).Token); }
    catch (OperationCanceledException) { p.Kill(entireProcessTree: true); throw; }
    var log = ((await stdout) + (await stderr)).Replace(staging + Path.DirectorySeparatorChar, "")
        .Replace(output + Path.DirectorySeparatorChar, "").Trim();
    File.WriteAllText(Path.Combine(output, $"{++total:00}-{project}.log"), log);
    if (p.ExitCode != exit || !log.Contains(required, StringComparison.Ordinal))
        throw new Exception($"{label}: exit={p.ExitCode}, expected={exit}\n{log}");
    if (!groups.TryGetValue(number, out var reports)) groups.Add(number, reports = []);
    reports.Add((label, log));
    Console.WriteLine($"\n=== {label} ===");
    Console.WriteLine(log);
    Console.WriteLine($"PASS: exit={p.ExitCode}, expected={exit}");
}
static string FindRoot()
{
    for (var d = new DirectoryInfo(AppContext.BaseDirectory); d is not null; d=d.Parent)
        if (File.Exists(Path.Combine(d.FullName,"ServiceDesk.sln"))) return d.FullName;
    throw new DirectoryNotFoundException("ServiceDesk.sln not found");
}
static string H(string s) => WebUtility.HtmlEncode(s);
static int GetFreePort()
{
    var listener = new TcpListener(IPAddress.Loopback, 0);
    listener.Start();
    try { return ((IPEndPoint)listener.LocalEndpoint).Port; }
    finally { listener.Stop(); }
}
```

ПРИЛОЖЕНИЕ Б

Фактический вывод контрольного прогона

Я проверил сохранение и отправку заявки, установку версии 1.1.0, отклонение изменённого пакета, разграничение прав, сертификаты и DPAPI. Резервная копия содержит четыре записи архива. Общий запуск завершил 13 сценариев с ожидаемыми кодами и результатами.

Снимки выполнения программы в Visual Studio

Я запустил программы в Visual Studio под Windows. На снимках приведены исходный код и результаты контрольного запуска.

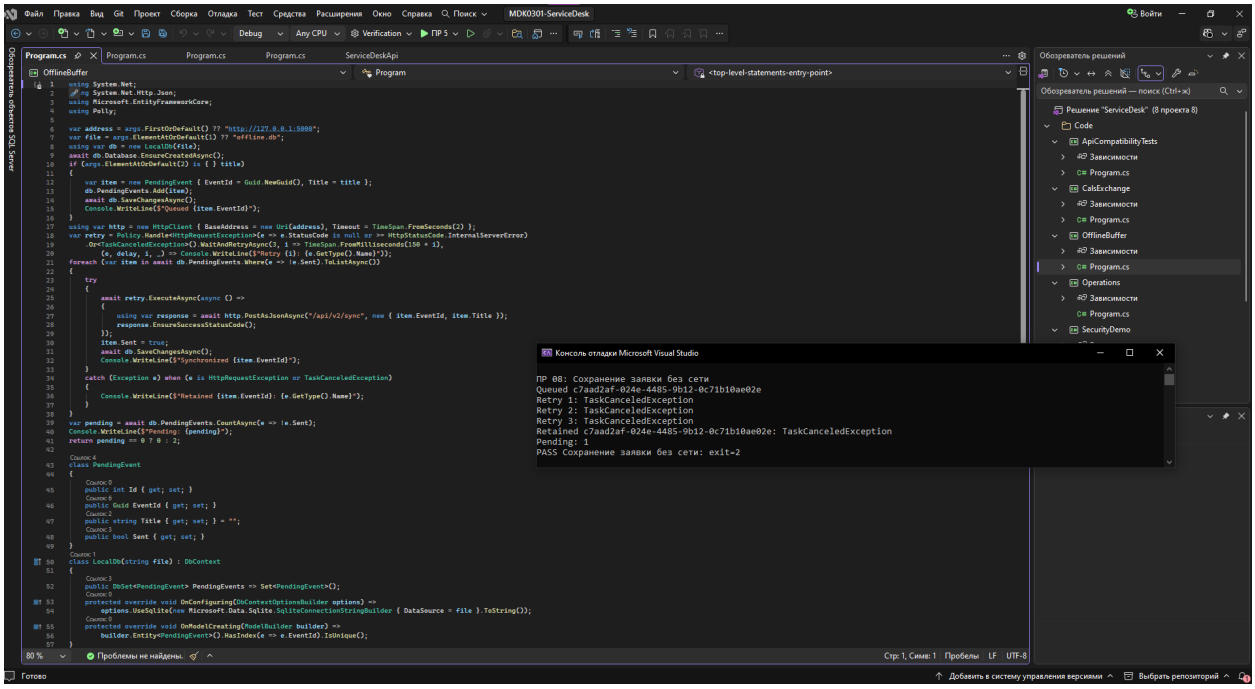


Рисунок Б.1 — Сохранение заявки при недоступном соединении. После трёх повторных попыток заявка осталась в очереди, Pending: 1.

Код завершения 2 означает, что отправка не выполнена и запись сохранена для повторной доставки. Для сценария отсутствия соединения это ожидаемый результат.

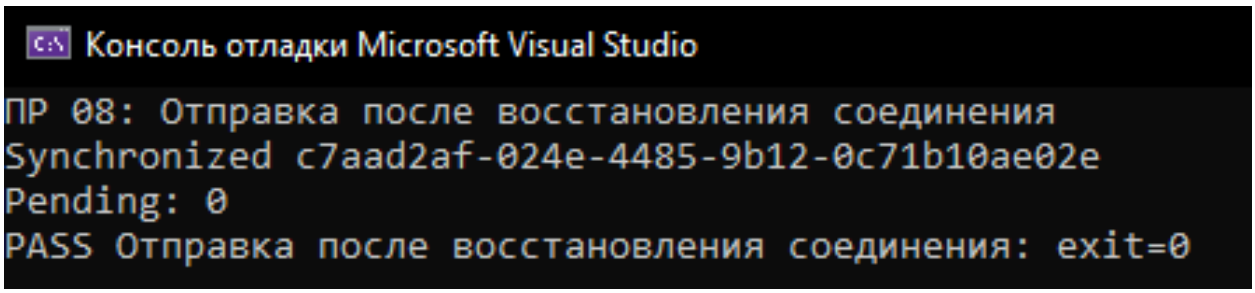


Рисунок Б.2 — Отправка той же заявки после восстановления соединения. Очередь пуста, Pending: 0.

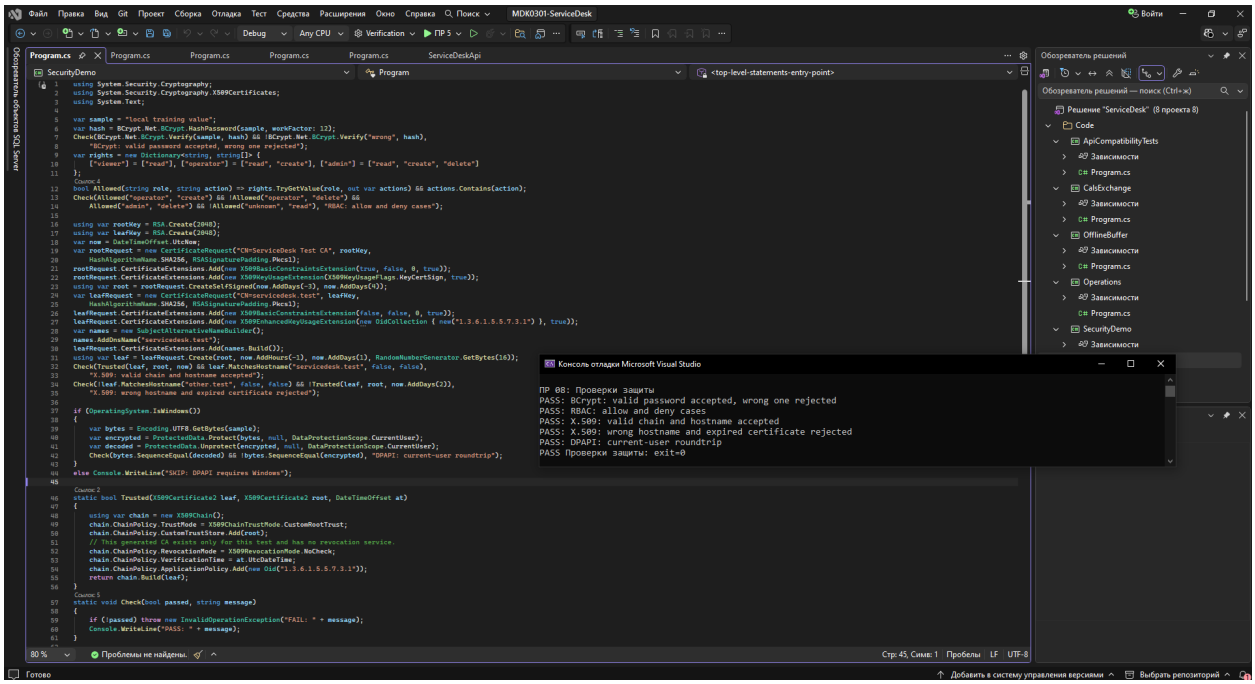


Рисунок Б.3 — Проверки BCrypt, разграничения прав RBAC, сертификатов X.509 и защиты данных DPAPI в Windows.

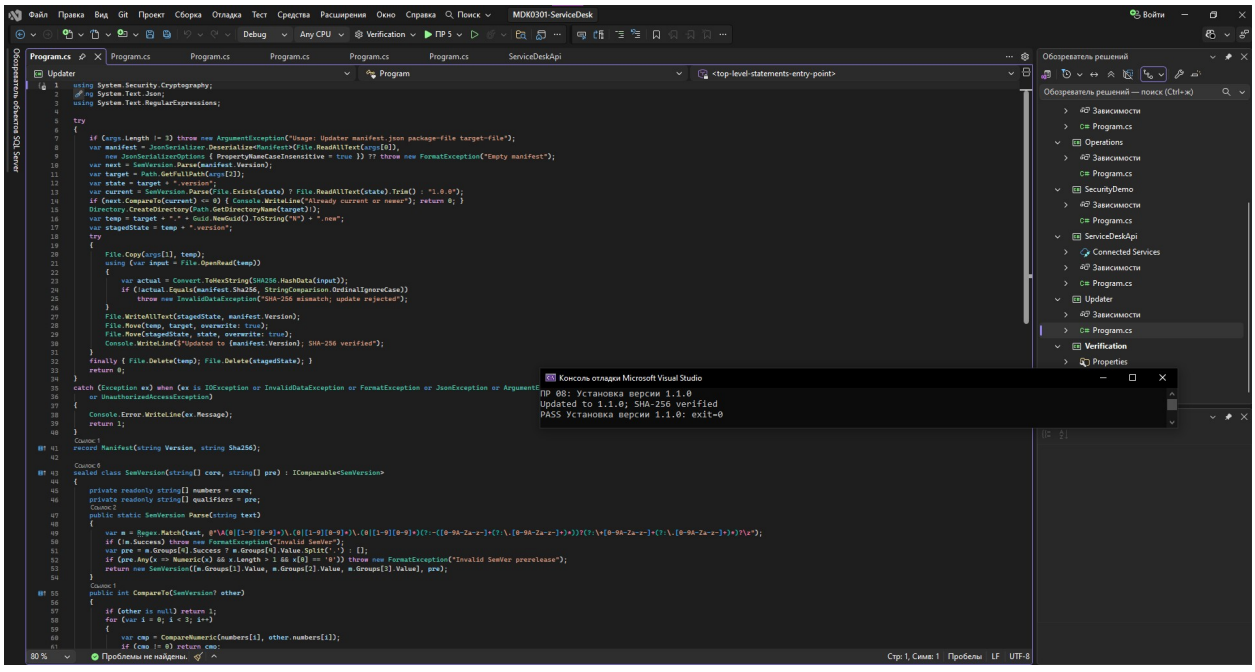


Рисунок Б.4 — Установка версии 1.1.0 после успешной проверки контрольной суммы SHA-256.

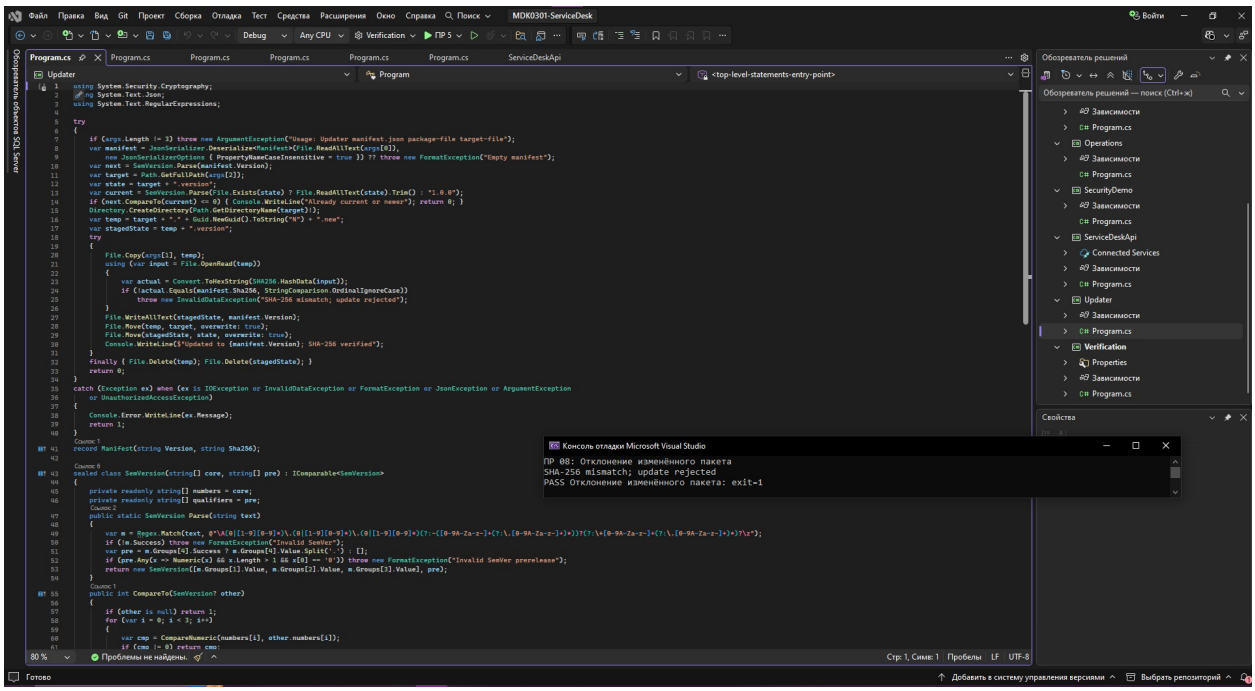


Рисунок Б.5 — Отклонение изменённого пакета при несовпадении SHA-256.

Код завершения 1 в отрицательном сценарии означает отклонение изменённого пакета. Проверяющая программа подтвердила ожидаемый отказ строкой PASS.

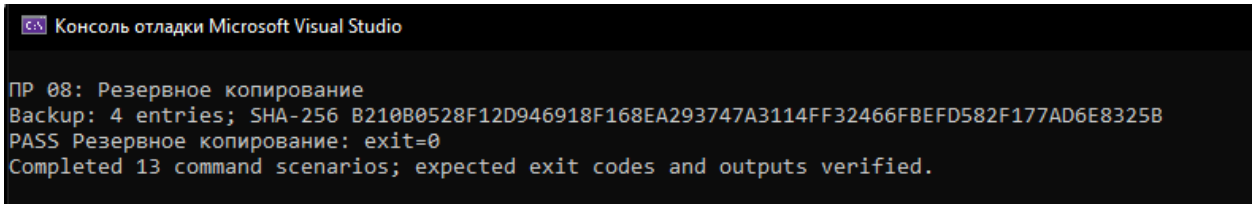


Рисунок Б.6 — Резервное копирование и итог контрольного запуска. Завершены 13 сценариев с ожидаемыми результатами.